

**FORMAN CHRISTIAN COLLEGE
(A CHARTERED UNIVERSITY)**



COMP 410 A

SP24

Project: SOS Game with Multiprocessing

Daim Bin Khalid – 251686775

2nd June 2024

INTRODUCTION

In this project, I've developed the SOS game and employed parallel programming using multiprocessing python library to determine the winner. The SOS game is a multiplayer game played on a grid, where participants alternate placing 'S' or 'O' symbols on the grid. The objective is to form sequences of 'SOS' either horizontally, vertically, or diagonally. The player who scores an 'SOS' first wins the game. If the grid is filled and then the game comes to a draw. The grid made is 8x8 as instructed in the manual.

IMPLEMENTATION

The project was implemented in two stages, a sequential version of the game and a multiprocessing version to compare how each of them perform. This is to gain a sense of effectiveness of multiprocessing being used and if it belongs in implementing a grid-based SOS game.

Sequential

Program flow:

- Implemented an SOS class with functions to be able enter an 'S' or an 'O'
- Have two players in the game and switch them after each turn.
- Check for grid array being full for draw condition.
- Check for 'SOS' winning conditions horizontally, vertically, and diagonally.
- Wrap the SOS class in a GUI class to emulate the grid and click actions.

Win Conditions Implementation:

```
def checkWinConditions(board):
```

Args: numpy grid array

Functionality: compare row and column positions to check for horizontal, vertical, and diagonal conditions.

Winning conditions occurring horizontally:

```
# win condition in rows eg. S | O | S
for i in range(start_row, end_row):
    for j in range(max(0, start_col - 2), min(cols - 2, end_col)):
        if board[i][j:j + 3] == ['S', 'O', 'S']:
            sosTrue += 1
```

Winning conditions occurring vertically:

```
# win condition in cols (vertical)
for j in range(start_col, end_col):
    for i in range(max(0, start_row - 2), min(rows - 2, end_row)):
        if [board[i+k][j] for k in range(3)] == ['S', 'O', 'S']:
            sosTrue += 1
```

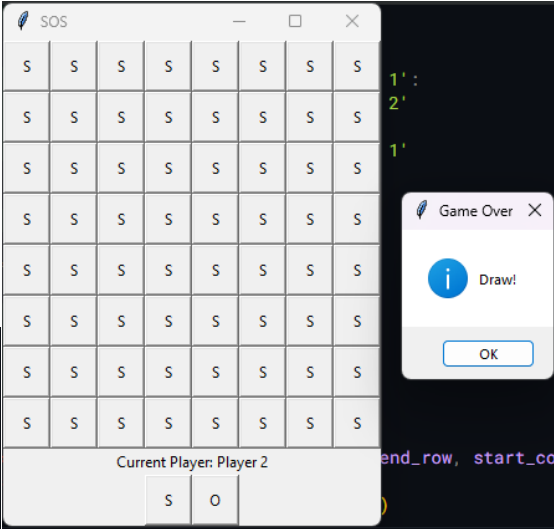
Winning conditions occurring diagonally:

```
# win condition diagonally in either direction
for i in range(max(0, start_row - 2), min(rows - 2, end_row)):
    for j in range(max(0, start_col - 2), min(cols - 2, end_col)):
        if (board[i][j] == 'S' and board[i + 1][j + 1] == 'O' and board[i + 2][j + 2] == 'S'):
            sosTrue += 1
        if (board[i][j + 2] == 'S' and board[i + 1][j + 1] == 'O' and board[i + 2][j] == 'S'):
            sosTrue += 1
```

Draw Condition Implementation:

Game ends at a draw if all the grid is filled and no player has won yet.

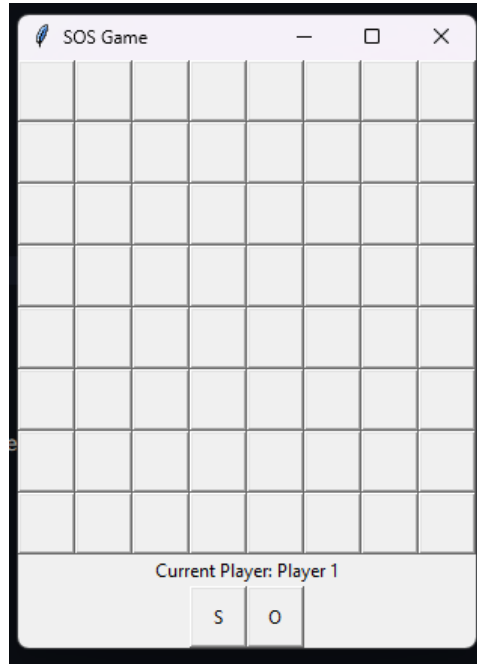
```
def isBoardFull(self):
    for row in self.board:
        if ' ' in row:
            return False
    return True
```



The screenshot shows a game window titled 'SOS' with a 9x9 grid. All cells in the grid contain the letter 'S'. A 'Game Over' dialog box is open, displaying an information icon, the text 'Draw!', and an 'OK' button. Below the grid, it says 'Current Player: Player 2' and shows a small 3x3 grid with 'S' and 'O'.

GUI Implementation:

- Using tkinter library to create a GUI showing current player, grid and 'S' or 'O' selection buttons.



- 8x8 grid buttons to click 'S' and 'O' with.

```
self.buttons = [[None for _ in range(8)] for _ in range(8)]
```

```
for i in range(8):
    for j in range(8):
        button = tk.Button(self.master, text=' ', width=4, height=2,
                           command=lambda row=i, col=j: self.onClick(row, col))
        button.grid(row=i, column=j)
        self.buttons[i][j] = button
```

- Reset grid when game is over by winning or by draw:

```
def reset(self):
    self.game = SOS()
    for i in range(8):
        for j in range(8):
            self.buttons[i][j].config(text=' ')
    self.label.config(text="Current Player: Player 1")
```

Multiprocessing Implementation

The calculation done in checkWinConditions function is done through nested loops cycling through the grid. Using multiprocessing, this function is wrapped in parallelCheckWinConditions function that uses a start and end point of a section of the grid instead of the entirety of it so that each process can calculate a separate section of the grid.

The checkWinConditions is wrapped within a range defining function which is then called by the pool of processes individually.

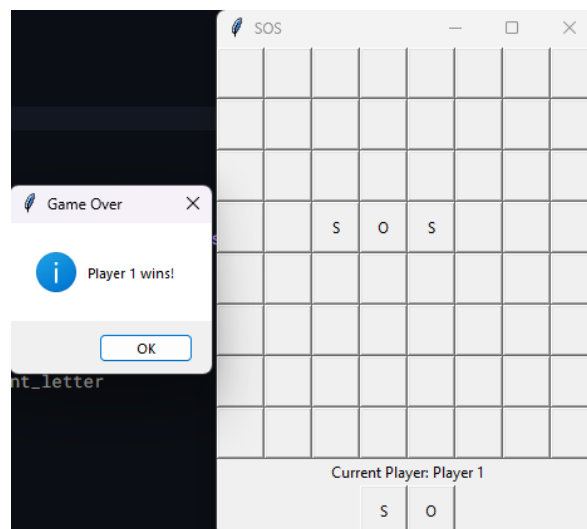
```
def checkWinRange(args):
    board, start_row, end_row, size = args
    return checkWinConditions(board, start_row, end_row, 0, size)
```

```
def parallelCheckWinConditions(board, size, num_processes=cpu_count()):
    pool = Pool(num_processes)
    rows_per_process = size // num_processes
    ranges = [(board, i * rows_per_process, (i + 1) * rows_per_process, size) for i in range(num_processes)]
    # ensure the last range includes any remaining rows
    ranges[-1] = (board, ranges[-1][1], size, size)
    results = pool.map(checkWinRange, ranges)
    pool.close()
    pool.join()
    return sum(results)
```

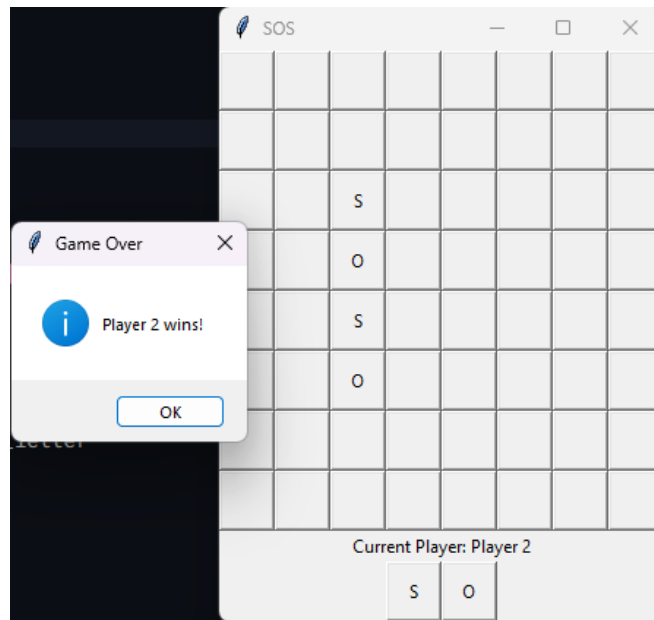
Benefits of making the game calculate this over multiprocessing is to make the responsiveness of getting a win condition check more efficient.

Sample Runs

Horizontal Win Condition Example:



Vertical Win Condition Example:



Diagonal Win Condition Example:

